

HelixOTA

HelixOTA

Universelle, entkoppelte Over-the-Air-Updates - von Grund auf „unbrickbar“ konzipiert.

Zusammenfassung

Helix OTA ist ein universelles, tiefgreifend entkoppeltes Over-the-Air-Update-System: eine Go-Steuerungsebene kombiniert mit betriebssystemspezifischen Client-Agenten, entwickelt für sichere, schrittweise Firmware- und App-Updates in Flotten von einem einzelnen Board bis zu Millionen Geräten. Das erste Zielsystem ist Android 15 auf dem Orange Pi 5 Max.

Kurzbeschreibung

Helix OTA ist ein universelles Over-the-Air-Update-System – bestehend aus einer Go-Steuerungsebene und betriebssystemspezifischen Client-Agenten – das für null Systemkorruption, validierte Uploads und granulare, schrittweise Rollouts konzipiert wurde. Das erste Zielsystem ist Android 15 auf dem Orange Pi 5 Max, wobei Linux- und Windows-Adapter geplant sind.

Ausführliche Beschreibung

Helix OTA ist ein universelles, generisches und tiefgreifend entkoppeltes Over-the-Air-(OTA)-Update-System, das auf einem einzigen unnachgiebigen Versprechen basiert: Ein Update darf ein funktionierendes Gerät niemals in einen „Brick“ verwandeln. Es besteht aus einem Go-Server als **Steuerungsebene**, betriebssystemspezifischen **SDKs/Agenten** und einem **Verwaltungs-Dashboard**. Das System ist von Grund auf so konzipiert, dass es durch austauschbare Betriebssystem-Adapter in *jedes* Betriebssystem integriert werden kann – statt für jede Plattform neu entwickelt werden zu müssen. Das erste Ziel ist

Android 15 (alle Varianten) auf dem Orange Pi 5 Max, wo die Build-Pipeline Flash-Images zusammen mit einem validierten OTA-.zip-Archiv und obligatorischen Hash-Dateien erzeugt, sodass kein Artefakt ohne verifizierbaren Fingerabdruck auf ein Gerät gelangt. Linux, Windows und andere Betriebssysteme folgen auf der Roadmap hinter derselben Adapter-Schnittstelle – sie benötigen lediglich ihren Adapter, keine Neuentwicklung.

Das Design basiert auf vom Betreiber definierten, unverhandelbaren Architekturprinzipien: null Systemkorruption, obligatorische Validierung jedes Artefakts vor dem Deployment, granulare Rollouts (entweder sofort oder schrittweise mit 5/10/30...100 % und Stopp-Fortschritts-Kontrolle), vollständige Transparenz der Geräteflotte sowie lineare Skalierbarkeit von einem einzelnen Board auf dem Labortisch bis zu Millionen Geräten im Feld. Die festgelegte Architektur kombiniert geräteseitige native Android-A/B-Updates – AOSP `update_engine` mit AVB/dm-verity und automatischem Rollback bei Boot-Fehlern – mit einer maßgeschneiderten, entkoppelten Go-Steuerungsebene. Dadurch liegt die Sicherheit sowohl in der bootnahen Hardware-Ebene *als auch* auf dem Server, nicht in einer einzigen anfälligen Schicht. Zwei Schnittstellen sind bewusst herauslösbar gestaltet: eine Betriebssystem-Adapter-Schnittstelle, die echte Universalität verspricht, und eine Rollout-Engine-Schnittstelle, die schrittweise Updates betriebssystemunabhängig macht. Das gesamte System ist in sechs öffentliche, unabhängig versionierte `ota-*`-Submodule unterteilt – wiederverwendbare Bausteine statt eines Monolithen.

Helix OTA befindet sich derzeit in einer Spezifikations-, Forschungs- und Testabdeckungsphase. Das Repository enthält den verbindlichen Design-Korpus, die Dokumentations-Export-Pipeline und das Submodul-Grundgerüst. Gemäß seiner „Anti-Bluff“-Governance ist explizit festgehalten, dass ein fertiger Produktionsserver und -Agent noch nicht existieren. Was heute ausgeliefert wird, ist der Bauplan und sein Grundgerüst – ehrlich als solches gekennzeichnet.

Warum wir es entwickelt haben

OTA wird normalerweise für jedes Gerät und jedes Betriebssystem neu erfunden, und ein fehlerhaftes Update kann eine ganze Flotte unbrauchbar machen. Helix OTA wurde als universelles, sicherheitsorientiertes Update-System konzipiert, das jedes Betriebssystem über Adapter übernehmen kann – mit Rollback- und Validierungsgarantien, die von Anfang an in die Architektur integriert sind, statt nachträglich hinzugefügt zu werden.

Warum es ein Game-Changer ist

Es weigert sich, „Geräte niemals unbrauchbar machen“ und „Updates schrittweise und beobachtbar ausrollen“ als Best-Effort-Features zu behandeln, die man sich nur wünscht – stattdessen sind es architektonische Invarianten, die sowohl im Boot-Pfad als auch in der Steuerungsebene verankert sind. Und indem die Ausroll-Engine und die Betriebssystem-Schicht als austauschbare Schnittstellen statt als fest verdrahtete Annahmen gestaltet werden, kann dieselbe Steuerungsebene heute Android steuern und ist bereit, später andere Betriebssysteme zu unterstützen – einfach durch Hinzufügen eines Adapters, ohne Fork, ohne Neuschreiben, ohne die Sicherheitsgarantien neu erfinden zu müssen, denen man bereits vertraut.

Was innovativ ist

- **Zwei extrahierbare Schnittstellen** – eine OS-Adapter-Schnittstelle und eine betriebssystemunabhängige Ausroll-Engine – die „universell“ von einem Marketingbegriff zu einer strukturellen Eigenschaft des Codebase machen.
- **Mehrschichtige Sicherheit:** Geräteseitiges natives A/B (update_engine) + AVB/dm-verity + automatisches Rollback bei Boot-Fehlern, geschichtet *über* serverseitige Artefaktvalidierung – ein Update muss mehrere unabhängige Kontrollpunkte passieren, bevor es dauerhaft übernommen wird.
- **Katalogbasierte, entkoppelte** Aufteilung in sechs wiederverwendbare, unabhängig versionierte ota-*-Submodule, die man nach Bedarf nutzen kann, statt ein Monolith zu schlucken.
- **HTTP/3 (QUIC) als primäres Transportprotokoll** mit automatischem HTTP/2-Fallback und ausgehandelter Brotli/gzip-Komprimierung – moderne, latenzarme Auslieferung, die sich anpasst, statt zu versagen.
- **Anti-Bluff-Engineering:** Design und Status werden explizit als Spezifikationsphase gekennzeichnet, und nichts Unfertiges wird jemals als ausgeliefert behauptet – Ehrlichkeit als zentraler Ingenieurswert, nicht als Haftungsausschluss in den Fußnoten.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Garantie, dass ein fehlerhaftes Update ein Gerät niemals unbrauchbar macht** – das schwierigste Versprechen in OTA. Gelöst durch die Vorgabe eines geräteseitigen nativen Android-A/B: update_engine schreibt in den inaktiven Slot, während der

aktive Slot weiterläuft, AVB/dm-verity überprüft kryptografisch die Boot-Kette, und falls der neue Slot nicht startet, führt das Gerät automatisch ein Rollback durch – alles abgesichert durch obligatorische Vorabvalidierung der Artefakte, sodass ein beschädigtes Update bereits abgefangen wird, bevor es den Server verlässt.

- **Ein System für viele Betriebssysteme** – gelöst, indem Android-spezifische Annahmen nicht in den Kern integriert werden. Eine steckbare OS-Adapter-Schnittstelle kapselt plattformspezifische Details, und eine betriebssystemunabhängige Ausroll-Engine hält die Kampagnenlogik portabel, wobei beide als separate Submodule gehalten werden, sodass ein neues Betriebssystem eine Erweiterung darstellt, nie einen Eingriff in das Gesamtsystem.
- **Stufenweise, anhaltbare Ausrollungen** – gelöst mit einer dedizierten Ausroll-Engine, die in Prozent-Kohorten mit Erfolgs-/Fehler-Schwellenwerten und expliziter Stopp-/Weiter-Steuerung arbeitet, bewusst ohne HTTP-Kopplung, sodass dieselbe Engine Kampagnen unabhängig vom Transportprotokoll steuern kann.

Technologie-Stack

- **Go + Gin** — ausgewählt wegen des Nebenläufigkeitsmodells und des schlanken Deployment-Footprints; treibt die Kontrollebene, den Rollout-Engine und die Artefakt-Validatoren an und stellt die primäre Oberfläche REST /api/v1 bereit.
- **Kotlin/KMP** — gewählt, damit der Android-OTA-Agent auf dem Gerät Logik über verschiedene Ziele hinweg teilen kann; verwaltet den vollständigen Gerätezyklus: Abfrage / Download / Überprüfung / Anwendung / Bericht.
- **HTTP/3 (QUIC) → HTTP/2** — QUIC als primäres Transportprotokoll für latenzarme, robuste Auslieferung über instabile Mobilfunkverbindungen, mit automatischem HTTP/2-Fallback, sodass kein Gerät zurückgelassen wird; **Brotli/gzip** wird pro Anfrage ausgehandelt, um die Payloads zu verkleinern.
- **PostgreSQL** — ausgewählt für relationale Integrität im Gerätereister, bei Kampagnen und Telemetrie, wo die Korrektheit des Flottenstatus wichtiger ist als die reine Schreibgeschwindigkeit.
- **MinIO / S3** — als Artefakt-Blob-Speicher gewählt, damit große Firmware-Images in kostengünstigem Objektspeicher abgelegt werden können, entkoppelt von der relationalen Schicht.
- **AOSP update_engine + AVB/dm-verity + boot_control** — gewählt, weil die Wiederverwendung der bewährten Android-eigenen Virtual-A/B- und Verified-Boot-Mechanismen sicherer ist als die Entwicklung eines proprietären Updaters; steuert

Slot-Swaps und kryptografische Boot-Überprüfung auf dem Gerät.

- **React** — ausgewählt für das Verwaltungsdashboard, in dem Betreiber sich anmelden, Artefakte hochladen, Rollouts steuern und den Flottenstatus an einem Ort überwachen.
- **OpenTelemetry + Prometheus/Grafana** — gewählt für herstellerneutrale Instrumentierung; dient dazu, jeden Schritt eines Rollouts in Metriken und Dashboards sichtbar zu machen, statt ihn nur zu vermuten.

Status & Transparenzhinweise

- **Status: in Entwicklung.** Gemäß der projektinternen Anti-Bluff-Governance gibt es **noch keinen funktionierenden Produktionsserver oder -agenten** – dies ist eine Spezifikations-, Forschungs- und Testabdeckungsphase. Das Repository enthält den maßgeblichen Design-Korpus, die Dokumenten-Export-Pipeline und das Submodul-Gerüst.
- Die sechs öffentlichen, wiederverwendbaren Submodule (ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema) sind unter github.com/HelixDevelopment/ verfügbar.
- Testabdeckung und Latenzwerte im Repository sind die projektinternen, laufenden Aufzeichnungen und wurden nicht unabhängig bestätigt. Die in der README zitierten HelixConstitution-Klauselnummern sind UNBESTÄTIGT.
- **Lizenz: Apache-2.0.**

Prioritätsstufe: Helix-primär.